

AcquirerX Deep-Dive 5/10 — Terminal Service

Service: `terminal-service` **Port:** `9092` **Database:** `acquirerx_terminal` **Role:** Manages POS terminals, their configuration profiles, and health telemetry

1. What It Is

A **terminal** is a physical POS device — what the cashier swipes/taps your card on. Each terminal is registered in the system, assigned to a store (which is owned by a merchant), configured with parameters (currency, language, capabilities), and continuously reports its health.

Terminal-service owns three concerns:

1. **Terminal entity** — registration, capability, status, store/merchant linkage
2. **ParamProfile** — versioned configuration that can be assigned to terminals
3. **TerminalHealth** — telemetry: battery, signal strength, firmware version, last-seen timestamp

Without a registered ACTIVE terminal, no transaction can be authorized. Without recent health data, ops can't tell if a terminal is alive.

2. Why It's a Separate Service

- **Different lifecycle** — terminal records are created/updated rarely; health data is high-frequency telemetry
 - **Different access patterns** — admins manage terminals; the network layer pushes health updates
 - **Independent scaling** — health updates can be a high-write workload that shouldn't affect merchant CRUD
 - **Domain ownership** — terminal-specific logic (capability matching, parameter versioning) is contained here
-

3. Tech Stack

- Spring Boot 3.x

- Spring Data JPA + MySQL
 - Spring Cloud OpenFeign — calls merchant-service
 - Resilience4J — circuit breaker on Feign calls
 - Lombok
 - Bean Validation
-

4. Code Structure

```
terminal-service/src/main/java/com/acquirerx/terminal/
├─ TerminalServiceApplication.java
├─ client/
│   ├─ MerchantServiceClient.java
│   └─ MerchantServiceClientFallback.java
├─ terminal/
│   ├─ controller/
│   │   ├─ TerminalController.java
│   │   └─ TerminalProvisioningController.java
│   ├─ dto/ (Terminal, ParamProfile, Health, Stats DTOs)
│   ├─ entity/
│   │   ├─ Terminal.java
│   │   ├─ ParamProfile.java
│   │   └─ TerminalHealth.java
│   └─ service/
│       ├─ TerminalService.java
│       └─ TerminalProvisioningService.java
```

5. Entities

5.1 `Terminal` (table: `terminal`)

Field	Type	Purpose
<code>terminalId</code>	Long PK	Auto-increment internal ID
<code>tid</code>	String, unique	Public Terminal ID — 8-digit string visible on POS receipts
<code>brandModel</code>	String	e.g., "Verifone V200c", "Ingenico Move 5000"

Field	Type	Purpose
<code>capability</code>	String	MAGSTRIPE / EMV / CTLS (contactless) / EMV_CTLS / etc.
<code>status</code>	String	PENDING (just registered) → ACTIVE → INACTIVE
<code>storeId</code>	Long FK	Store the terminal belongs to
<code>merchantId</code>	Long FK	Merchant (denormalized for fast queries — saves a JOIN)
<code>paramProfileId</code>	Long FK	Currently assigned parameter profile (nullable)
<code>createdAt</code>	LocalDateTime	

Why both `terminalId` (Long) and `tid` (String):

- `terminalId` — internal numeric ID, used in JPA, foreign keys, fast indexed lookups
- `tid` — public 8-digit terminal ID printed on receipts, visible to operators and merchants

This is the same dual-ID pattern used by Visa/Mastercard themselves. Internal IDs are integers for performance; public IDs are strings for human readability and uniqueness across networks.

Why `merchantId` is denormalized here: You can derive merchant from `terminal.store.merchant`, but that's a 3-level JOIN every time. Storing `merchantId` directly on terminal gives one-hop lookup. Trade-off: data duplication (merchant change requires updating all terminals), but merchant-store assignments rarely change after onboarding.

Capability values explained:

Code	Meaning
MAGSTRIPE	Old swipe-only — being phased out
EMV	Chip card insert — secure, current standard
CTLS	Contactless (NFC) — tap to pay
EMV_CTLS	Both chip and contactless
ALL	Magstripe + EMV + CTLS

This drives auth logic — if a terminal is MAGSTRIPE-only, it can't process contactless transactions.

5.2 ParamProfile (table: param_profile)

A reusable bundle of configuration that can be assigned to many terminals.

Field	Type	Purpose
paramProfileId	Long PK	
name	String	Human-readable, e.g., "Standard India v3"
paramsJson	TEXT	JSON blob of actual parameters
version	Integer	Increments on each update — enables versioning
status	String	ACTIVE / INACTIVE
createdAt, updatedAt		

Example paramsJson:

```
json
{
  "currency": "INR",
  "language": "en-IN",
  "merchantTimeout": 30,
  "receiptHeader": "TechMart Retail",
  "supportedNetworks": ["VISA", "MASTERCARD", "RUPAY"],
  "tipPromptEnabled": false
}
```

Why JSON-as-string (not separate columns): Parameters vary between regions, terminal models, and merchant types. Modeling every possible param as a column would lead to a hundred-column table where 90% are null for any given profile. JSON gives flexibility, with the trade-off of less queryability (you can't `WHERE currency = 'INR'` easily).

Why versioning: When a profile is updated, the `version` increments. Terminals can choose to download the new version on next heartbeat. This avoids broken terminals if a bad config is pushed — you can roll back to an older version.

5.3 `TerminalHealth` (table: `terminal_health`)

Field	Type	Purpose
<code>healthId</code>	Long PK	
<code>terminalId</code>	Long FK	
<code>lastSeen</code>	LocalDateTime	Last heartbeat timestamp
<code>batteryPct</code>	Integer	0-100 (null for AC-powered terminals)
<code>signalStrength</code>	Integer	Signal bars (0-5) for cellular/WiFi
<code>status</code>	String	ONLINE / OFFLINE / DEGRADED
<code>firmwareVersion</code>	String	e.g., "v3.2.1"
<code>ipAddress</code>	String	Last-known IP
<code>updatedAt</code>	LocalDateTime	

Health status values:

- ONLINE — recent heartbeat (within ~5 minutes)
- OFFLINE — no heartbeat for >X minutes
- DEGRADED — heartbeats coming but battery low or signal weak

Why a separate table (not columns on Terminal): Health is high-write data — every terminal pushes updates frequently. Storing it on Terminal would mean every health update triggers a row update on the heavily-read Terminal table, causing lock contention. Splitting them lets the read-heavy `terminal` table stay fast and the write-heavy `terminal_health` table optimize separately.

6. The TerminalService

Manages the Terminal entity. CRUD, search, status, stats.

6.1 Typical method shape

```
java
```

```

@Service
@RequiredArgsConstructor
public class TerminalService {

    private static final Set<String> SORTABLE_FIELDS = Set.of(
        "terminalId", "tid", "brandModel", "capability", "status", "createdAt"
    );

    private final TerminalRepository terminalRepository;
    private final MerchantServiceClient merchantClient;

    public TerminalResponseDTO createTerminal(Long storeId, TerminalRequestDTO dto) {
        // Verify store exists locally? No – store is in merchant-service.
        // Instead, fetch merchant via Feign to validate.
        // Or: trust the storeId, denormalize merchantId by calling merchant-service

        Long merchantId = resolveMerchantIdForStore(storeId);

        if (terminalRepository.existsByTid(dto.getTid())) {
            throw new IllegalStateException("TID already in use: " + dto.getTid());
        }

        Terminal t = new Terminal();
        t.setTid(dto.getTid());
        t.setBrandModel(dto.getBrandModel());
        t.setCapability(dto.getCapability());
        t.setStoreId(storeId);
        t.setMerchantId(merchantId);
        t.setStatus("PENDING");

        return toResponse(terminalRepository.save(t));
    }

    public TerminalResponseDTO getById(Long id) {
        return terminalRepository.findById(id)
            .map(this::toResponse)
            .orElseThrow(() -> new ResourceNotFoundException("Terminal not found: "
        // ... search, getByStore, updateStatus, getStats
    }

```

Key decisions:

- **TID uniqueness check** — explicit before insert (clearer error than DB constraint exception)
- **merchantId denormalization on create** — fetch from merchant-service via Feign once, store locally
- **Status starts at PENDING** — operations team activates after physical install
- **Sortable field whitelist** — same defensive pattern as merchant-service

6.2 The Feign call to merchant-service

```
java

@FeignClient(name = "MERCHANT-SERVICE", fallback = MerchantServiceClientFallback.class)
public interface MerchantServiceClient {

    @GetMapping("/api/v1/merchants/{id}")
    Map<String, Object> getMerchantById(@PathVariable Long id);

    // Optionally: getStoreById, getMerchantByStoreId, etc.
}
```

```
java

@Component
public class MerchantServiceClientFallback implements MerchantServiceClient {

    @Override
    public Map<String, Object> getMerchantById(Long id) {
        // Safe default – empty map. Caller checks for missing data.
        return Collections.emptyMap();
    }

}
```

Why the fallback returns empty (not throws): Terminal creation can technically proceed without immediate merchant enrichment — the `merchantId` is supplied by the caller. The Feign call is for *extra context*, so a graceful empty response is acceptable. If the data is critical, the calling code checks for it explicitly.

In transaction-service, the equivalent fallback for `TerminalServiceClient` *throws*, because terminal info IS critical to authorize a transaction.

7. The TerminalProvisioningService

Manages ParamProfile and TerminalHealth.

7.1 ParamProfile management

```
java

public ParamProfileResponseDTO createProfile(ParamProfileRequestDTO dto) {
    ParamProfile p = new ParamProfile();
    p.setName(dto.getName());
    p.setParamsJson(dto.getParamsJson());
    p.setVersion(1); // first version
    p.setStatus("ACTIVE");
    return toProfileResponse(paramProfileRepo.save(p));
}

public ParamProfileResponseDTO updateProfile(Long profileId, ParamProfileRequestDTO
    ParamProfile p = paramProfileRepo.findById(profileId).orElseThrow(...);
    p.setParamsJson(dto.getParamsJson());
    p.setVersion(p.getVersion() + 1); // bump version
    return toProfileResponse(paramProfileRepo.save(p));
}

public TerminalResponseDTO assignProfile(Long profileId, Long terminalId) {
    ParamProfile p = paramProfileRepo.findById(profileId).orElseThrow(...);
    if (!"ACTIVE".equals(p.getStatus())) {
        throw new IllegalStateException("Cannot assign INACTIVE profile");
    }
    Terminal t = terminalRepo.findById(terminalId).orElseThrow(...);
    t.setParamProfileId(profileId);
    return toTerminalResponse(terminalRepo.save(t));
}
```

7.2 Health management

```
java
```



```

public TerminalHealthResponseDTO recordHealth(Long terminalId,
                                             Integer batteryPct,
                                             Integer signalStrength,
                                             String firmwareVersion,
                                             String ipAddress) {

    // upsert pattern – one health row per terminal
    TerminalHealth h = healthRepo.findByTerminalId(terminalId)
        .orElseGet(() -> {
            TerminalHealth fresh = new TerminalHealth();
            fresh.setTerminalId(terminalId);
            return fresh;
        });

    h.setLastSeen(LocalDateTime.now());
    h.setBatteryPct(batteryPct);
    h.setSignalStrength(signalStrength);
    h.setFirmwareVersion(firmwareVersion);
    h.setIpAddress(ipAddress);
    h.setStatus(determineStatus(batteryPct, signalStrength));

    return toHealthResponse(healthRepo.save(h));
}

private String determineStatus(Integer battery, Integer signal) {
    if (battery != null && battery < 20) return "DEGRADED";
    if (signal != null && signal < 2) return "DEGRADED";
    return "ONLINE";
}

public List<TerminalHealthResponseDTO> getStaleTerminals() {
    LocalDateTime cutoff = LocalDateTime.now().minusMinutes(10);
    return healthRepo.findByLastSeenBefore(cutoff).stream()
        .map(this::toHealthResponse)
        .toList();
}

public List<TerminalHealthResponseDTO> getLowBatteryTerminals() {
    return healthRepo.findByBatteryPctLessThan(20).stream()
        .map(this::toHealthResponse)
        .toList();
}

```

Upsert pattern: A terminal has exactly one current health row. New telemetry overwrites the previous values. If you wanted full history, you'd use a separate table (`terminal_health_history`) with append-only writes — typical for time-series data, but AcquirerX keeps it simple with one row.

Status derivation: The status field is computed from raw metrics (battery, signal). This keeps the rule logic in one place; clients just read `status` instead of recomputing it.

Stale detection: Operations needs to know which terminals haven't checked in. `getStaleTerminals` returns terminals whose `lastSeen` is older than 10 minutes — likely offline or networking issue.

Low battery detection: For battery-powered terminals (typically used in restaurants for tableside payment), low battery is a critical alert. Returns all terminals with battery < 20%.

8. Repositories

8.1 `TerminalRepository`

```
java

public interface TerminalRepository extends JpaRepository<Terminal, Long> {
    boolean existsByTid(String tid);
    Page<Terminal> findByStoreId(Long storeId, Pageable pageable);
    Page<Terminal> findByMerchantId(Long merchantId, Pageable pageable);
    long countByStatus(String status);

    @Query("SELECT t FROM Terminal t WHERE " +
        "(:tid IS NULL OR t.tid = :tid) AND " +
        "(:brandModel IS NULL OR LOWER(t.brandModel) LIKE LOWER(CONCAT('%', :brandModel, '%'))" +
        "(:capability IS NULL OR t.capability = :capability) AND " +
        "(:status IS NULL OR t.status = :status) AND " +
        "(:storeId IS NULL OR t.storeId = :storeId) AND " +
        "(:merchantId IS NULL OR t.merchantId = :merchantId)")
    Page<Terminal> findByFiltersPaged(...);
}
```

8.2 `TerminalHealthRepository`

```
java
```

```
public interface TerminalHealthRepository extends JpaRepository<TerminalHealth, Long> {
    Optional<TerminalHealth> findByTerminalId(Long terminalId);
    List<TerminalHealth> findByLastSeenBefore(LocalDateTime cutoff);
    List<TerminalHealth> findByBatteryPctLessThan(Integer threshold);
    List<TerminalHealth> findByStatus(String status);
}
```

8.3 ParamProfileRepository

```
java

public interface ParamProfileRepository extends JpaRepository<ParamProfile, Long> {
    List<ParamProfile> findByStatus(String status);
}
```

9. Controllers

9.1 TerminalController.java — endpoints

```
GET    /terminals
GET    /terminals/{id}
POST   /stores/{storeId}/terminals
PUT    /terminals/{id}
DELETE /terminals/{id}
POST   /terminals/search
GET    /stores/{storeId}/terminals
PATCH /terminals/{id}/status?newStatus=ACTIVE|INACTIVE|PENDING
GET    /terminals/stats
```

9.2 TerminalProvisioningController.java — endpoints

```
GET    /terminals/provisioning/profiles
GET    /terminals/provisioning/profiles/active
POST   /terminals/provisioning/profiles
PUT    /terminals/provisioning/profiles/{profileId}
PATCH /terminals/provisioning/profiles/{profileId}/deactivate
POST   /terminals/provisioning/profiles/{profileId}/assign/{terminalId}

GET    /terminals/provisioning/health
```

```
GET    /terminals/provisioning/health/{terminalId}
POST   /terminals/provisioning/health/{terminalId}?
batteryPct=&signalStrength=&firmwareVersion=&ipAddress=
GET    /terminals/provisioning/health/status/{status}
GET    /terminals/provisioning/health/stale
GET    /terminals/provisioning/health/low-battery
```

Why `/provisioning/` segment: Provisioning groups all the "device-management" concerns (profiles, health) under one namespace, distinct from the core terminal CRUD. Makes the API self-documenting and lets you swagger-tag them separately.

10. Used By (Other Services)

Service	When	Endpoint used
transaction-service	Every authorize/void/refund — needs to fetch terminal info (TID, merchantId, store)	<code>GET /terminals/{id}</code>
risk-service (indirectly)	Risk check uses <code>tid</code> from terminal	Receives via transaction-service

Transaction-service is the heaviest caller — every transaction triggers a Feign call to terminal-service. This is why terminal-service has a `MerchantServiceClientFallback` that throws (without terminal info, no transaction can proceed).

11. The Feign Contract Other Services See

Defined in transaction-service (and others):

```
java
@FeignClient(name = "TERMINAL-SERVICE", fallback = TerminalServiceClientFallback.class)
public interface TerminalServiceClient {

    @GetMapping("/api/v1/terminals/{id}")
    Map<String, Object> getTerminalById(@PathVariable Long id);
}
```

The response is parsed as a generic `Map<String, Object>` rather than a typed DTO, because:

- Avoids needing to share DTO classes between services (no common JAR coupling)
- Caller picks fields it needs (`tid`, `merchantId`, `merchantName`)
- Loose coupling — terminal-service can add fields without breaking transaction-service

Trade-off: less type safety. The convention is "if you're in transaction-service, treat the response shape as a known contract documented in this deep-dive."

12. `application.properties`

properties

```
server.port=9092
spring.application.name=terminal-service

eureka.client.service-url.defaultZone=http://localhost:8761/eureka/

spring.datasource.url=jdbc:mysql://localhost:3306/acquirerx_terminal?createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=kail3114
spring.jpa.hibernate.ddl-auto=update

# Feign + Resilience4J
spring.cloud.openfeign.circuitbreaker.enabled=true
resilience4j.circuitbreaker.instances.default.sliding-window-size=10
resilience4j.circuitbreaker.instances.default.failure-rate-threshold=50
resilience4j.circuitbreaker.instances.default.wait-duration-in-open-state=30s
resilience4j.circuitbreaker.instances.default.permitted-number-of-calls-in-half-open-state=5
resilience4j.circuitbreaker.instances.default.minimum-number-of-calls=5
resilience4j.timelimiter.instances.default.timeout-duration=3s

server.servlet.context-path=/api/v1
```

13. The Full Provisioning Flow (End-to-End)

1. Operations creates a parameter profile
POST /terminals/provisioning/profiles

```
    { name: "Standard India v3", paramsJson: "{...}" }  
→ version=1, status=ACTIVE
```

2. New terminal is registered

```
POST /stores/{storeId}/terminals
```

```
    { tid: "12345678", brandModel: "Verifone V200c", capability: "EMV_CTLS" }  
→ status=PENDING, merchantId resolved via Feign call to merchant-service
```

3. Profile is assigned to the terminal

```
POST /terminals/provisioning/profiles/{profileId}/assign/{terminalId}
```

```
→ terminal.paramProfileId = profileId
```

4. Terminal is physically installed at the store

Operator manually changes status → ACTIVE

```
PATCH /terminals/{id}/status?newStatus=ACTIVE
```

5. Terminal starts heartbeating health

```
POST /terminals/provisioning/health/{terminalId}?
```

```
batteryPct=85&signalStrength=4&firmwareVersion=v3.2.1
```

```
→ upserts health row, status=ONLINE
```

6. Now ready: transactions can be authorized via this terminal

(in transaction-service, /transactions/authorize will call GET /terminals/{id} via Feign)

7. Ongoing:

- Health endpoint called every few minutes by the terminal
- Operations queries /provisioning/health/stale to find offline terminals
- Operations queries /provisioning/health/low-battery to find battery alerts
- When config needs to change, profile is updated → version bumps → terminal pulls on next heartbeat

14. Why This Design Is Robust

- **Health separated from terminal** — write-heavy telemetry doesn't slow down read-heavy CRUD
- **TID uniqueness enforced at code + DB** — no duplicates ever
- **Profile versioning** — safe rollback if a config push is bad
- **Capability flag** — drives transaction logic (no contactless attempts on a magstripe-only terminal)

- **Denormalized merchantId** — single hop for "who owns this terminal" queries
- **Resilience4J on Feign** — merchant-service downtime doesn't stop terminal CRUD (graceful degradation via fallback)

15. Common Pitfalls

Pitfall	Symptom	Fix
Forgetting to bump <code>version</code> on profile update	Terminals don't pull new config	Always increment in update method
Using JPA <code>@ManyToOne</code> <code>Store</code>	Cross-service join — fails because Store is in merchant-service	Use <code>Long storeId</code> instead
Hard delete of a terminal with txn history	Transactions reference dangling ID	Use status=INACTIVE (soft delete)
Health updates with no upsert logic	Duplicate rows per terminal	Use <code>findByTerminalId().orElseGet(new TerminalHealth())</code> pattern
Trusting <code>paramProfileId</code> is valid without checking ACTIVE	Inactive profile gets assigned	Validate profile.status before assignment

16. Summary

Aspect	Detail
Purpose	Terminals, parameter profiles, health
Port	9092
DB	<code>acquirersx_terminal</code> (3 tables)
Tech	Spring Boot, JPA, Feign, Resilience4J
Services	<code>TerminalService</code> , <code>TerminalProvisioningService</code>

Aspect	Detail
Controllers	2 (Terminal, TerminalProvisioning)
Calls	merchant-service (via Feign with fallback)
Called by	transaction-service (every txn), ops
Status machine	PENDING → ACTIVE → INACTIVE
Health states	ONLINE / OFFLINE / DEGRADED
Capabilities	MAGSTRIPE / EMV / CTLS / EMV_CTLS / ALL

Next: Transaction Service — switch (auth/void/refund), fee calculation, idempotency. This is the core of the whole system.